

DOCUMENT 5: IMPLEMENTATION.md

Master Integration Prompts for Antigravity IDE

Pre-Implementation Checklist

text

BEFORE STARTING, ensure you have:

- ☐ Backed up current codebase (git commit or branch)
- ☐ Created feature branch: git checkout -b ui-overhaul
- ☐ All existing pages load without errors
- ☐ Convex dev server runs: npx convex dev
- ☐ Next.js dev server runs: npm run dev
- ☐ Figma designs exported as reference images (screenshot each frame)
- ☐ Canva assets downloaded and placed in public/ directory

IMPLEMENTATION PROMPT 1: Design System Installation

Give this to Antigravity IDE first:

text

I am upgrading the UI of my Shinodroid security SaaS app.

CRITICAL RULES – FOLLOW THESE AT ALL TIMES:

1.  NEVER modify ANY file inside the convex/ directory
2.  NEVER change the logic of any useQuery, useMutation, or useAction calls
3.  NEVER alter ConvexClientProvider.tsx or auth flow logic
4.  NEVER change route paths or file locations in app/
5.  ONLY modify: TSX component markup, CSS styles, visual layout
6.  You may ADD new component files in a src/components/ directory
7.  You may ADD new utility files in src/lib/
8.  You may INSTALL new npm packages for UI only

STEP 1: Install required packages.

Run these commands:

```
npm install framer-motion recharts react-dropzone lucide-react clsx tailwind-merge
```

STEP 2: Update tailwind.config.ts

Extend (don't replace) the existing config with these additions:

[PASTE THE ENTIRE tailwind.config.ts EXTENSION FROM DOCUMENT 1]

STEP 3: Add extended design tokens to globals.css

Open src/app/globals.css. KEEP all existing CSS. ADD the following new custom properties inside the existing :root block:

[PASTE THE ENTIRE CSS CUSTOM PROPERTIES SECTION FROM DOCUMENT 1]

Also ADD all the @keyframes and utility classes from Document 1 AFTER the existing CSS rules.

STEP 4: Create a utility file src/lib/utls.ts:

```
import { clsx, type ClassValue } from 'clsx'
import { twMerge } from 'tailwind-merge'

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}

export function getScoreTier(score: number) {
  if (score >= 86) return { tier: 'excellent', grade: 'A', color: 'var(--gradient-score-excellent)' }
  if (score >= 71) return { tier: 'good', grade: 'B', color: 'var(--gradient-score-good)' }
  if (score >= 51) return { tier: 'medium', grade: 'C', color: 'var(--gradient-score-medium)' }
  if (score >= 31) return { tier: 'high', grade: 'D', color: 'var(--gradient-score-high)' }
  return { tier: 'critical', grade: 'F', color: 'var(--gradient-score-critical)' }
}

export function getScoreGlowClass(score: number) {
  if (score >= 86) return 'shadow-[0_0_20px_rgba(16,185,129,0.2)]'
  if (score >= 71) return 'shadow-[0_0_20px_rgba(59,130,246,0.2)]'
  if (score >= 51) return 'shadow-[0_0_20px_rgba(245,158,11,0.2)]'
  if (score >= 31) return 'shadow-[0_0_20px_rgba(249,115,22,0.2)]'
  return 'shadow-[0_0_20px_rgba(239,68,68,0.2)]'
}
```

```

export function getSeverityStyles(severity: string) {
  const map: Record<string, { bg: string; text: string; border: string; dot:
string }> = {
    critical: {
      bg: 'bg-[rgba(220,38,38,0.10)]',
      text: 'text-[#ef4444]',
      border: 'border-[rgba(220,38,38,0.25)]',
      dot: 'bg-[#ef4444]',
    },
    high: {
      bg: 'bg-[rgba(239,68,68,0.08)]',
      text: 'text-[#f87171]',
      border: 'border-[rgba(239,68,68,0.20)]',
      dot: 'bg-[#f97316]',
    },
    medium: {
      bg: 'bg-[rgba(245,158,11,0.08)]',
      text: 'text-[#fbbf24]',
      border: 'border-[rgba(245,158,11,0.20)]',
      dot: 'bg-[#f59e0b]',
    },
    low: {
      bg: 'bg-[rgba(59,130,246,0.08)]',
      text: 'text-[#60a5fa]',
      border: 'border-[rgba(59,130,246,0.20)]',
      dot: 'bg-[#3b82f6]',
    },
    info: {
      bg: 'bg-[rgba(156,163,176,0.08)]',
      text: 'text-[#9ca3b0]',
      border: 'border-[rgba(156,163,176,0.20)]',
      dot: 'bg-[#6b7280]',
    },
  }
  return map[severity.toLowerCase()] || map.info
}

```

```

export function formatTimeAgo(date: Date | number): string {
  const now = Date.now()
  const timestamp = typeof date === 'number' ? date : date.getTime()
  const seconds = Math.floor((now - timestamp) / 1000)

  if (seconds < 60) return 'just now'
  if (seconds < 3600) return `${Math.floor(seconds / 60)}m ago`
  if (seconds < 86400) return `${Math.floor(seconds / 3600)}h ago`
}

```

```

    if (seconds < 604800) return `${Math.floor(seconds / 86400)}d ago`
    return new Date(timestamp).toLocaleDateString()
  }

export function formatDuration(ms: number): string {
  const minutes = Math.floor(ms / 60000)
  const seconds = Math.floor((ms % 60000) / 1000)
  return `${minutes}m ${seconds.toString().padStart(2, '0')}s`
}

export function formatFileSize(bytes: number): string {
  if (bytes < 1024) return bytes + ' B'
  if (bytes < 1048576) return (bytes / 1024).toFixed(1) + ' KB'
  return (bytes / 1048576).toFixed(1) + ' MB'
}

```

STEP 5: Create the component directory structure:

```

src/
├── components/
│   ├── ui/
│   │   ├── button.tsx
│   │   ├── input.tsx
│   │   ├── card.tsx
│   │   ├── badge.tsx
│   │   ├── tabs.tsx
│   │   ├── dropdown.tsx
│   │   ├── tooltip.tsx
│   │   ├── skeleton.tsx
│   │   ├── progress-bar.tsx
│   │   └── accordion.tsx
│   ├── security/
│   │   ├── score-gauge.tsx
│   │   ├── severity-bar.tsx
│   │   ├── finding-card.tsx
│   │   ├── pipeline-step.tsx
│   │   └── severity-badge.tsx
│   ├── layout/
│   │   ├── sidebar.tsx
│   │   ├── top-bar.tsx
│   │   ├── public-nav.tsx
│   │   └── footer.tsx
│   ├── charts/
│   │   ├── score-trend-chart.tsx
│   │   └── vuln-donut-chart.tsx
│   └── shared/

```

```
|   |─ empty-state.tsx
|   |─ error-state.tsx
|   |─ page-transition.tsx
```

Create ONLY the directory structure and empty files for now.
Don't write component code yet – we'll do each one step by step.

After completing all 5 steps, verify:

1. npm run dev still works without errors
2. All existing pages still render correctly
3. No Convex errors in console

Show me the updated globals.css and tailwind.config.ts when done.

IMPLEMENTATION PROMPT 2: Base UI Components

text

Now create the base UI components. These are the building blocks used across all pages. For each component:

- Use TypeScript with proper prop types
- Use the cn() utility from src/lib/utils.ts for class merging
- Use CSS custom properties from our design system
- Support forwarded refs where appropriate

COMPONENT 1: src/components/ui/button.tsx

```
import * as React from 'react'
import { cn } from '@lib/utils'
import { Loader2 } from 'lucide-react'
```

```
interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: 'primary' | 'secondary' | 'ghost' | 'danger' | 'success'
  size?: 'sm' | 'md' | 'lg'
  isLoading?: boolean
  leftIcon?: React.ReactNode
  rightIcon?: React.ReactNode
}
```

Create this component following these exact styles:

Primary: bg var(--accent), hover var(--accent-hover), active #6d28d9,
text white, disabled opacity 40%

Secondary: bg transparent, border 1px var(--border), hover bg var(--surface-3),

 hover border var(--border-strong), text var(--text-primary)

Ghost: bg transparent, hover bg var(--surface-3), text var(--text-secondary),
 hover text var(--text-primary)

Danger: bg var(--danger), hover #f87171, text white

Success: bg var(--success), hover #34d399, text white

Sizes:

 sm: h-8 px-3 text-xs gap-1.5 rounded-[var(--radius-sm)]

 md: h-10 px-4 text-sm gap-2 rounded-[var(--radius-sm)]

 lg: h-12 px-6 text-base gap-2 rounded-[var(--radius-sm)]

Loading: Replace children with <Loader2 className="animate-spin" />,
 disable button, keep width stable

All buttons: font-weight 500, transition colors var(--duration-fast) var(--ease-smooth),

 cursor-pointer, disabled:cursor-not-allowed

COMPONENT 2: src/components/ui/input.tsx

```
interface InputProps extends React.InputHTMLAttributes<HTMLInputElement> {  
  label?: string  
  error?: string  
  helperText?: string  
  leftIcon?: React.ReactNode  
  rightIcon?: React.ReactNode  
  inputSize?: 'sm' | 'md' | 'lg'  
}
```

Styles:

 bg var(--surface-1), border 1px var(--border), rounded var(--radius-sm),
 text var(--text-primary), placeholder var(--text-muted),
 focus: border var(--accent), ring 3px var(--border-interactive)
 error: border var(--danger), ring var(--danger) at 15%

 sm: h-9, md: h-10, lg: h-12

 padding: 8px 12px (+ icon padding if icons present)

Label: text-xs font-medium text-[var(--text-secondary)] mb-1.5

 uppercase tracking-wide

Error text: text-xs text-[var(--danger-text)] mt-1

Helper text: text-xs text-[var(--text-muted)] mt-1

COMPONENT 3: src/components/ui/card.tsx

```
interface CardProps extends React.HTMLAttributes<HTMLDivElement> {  
  variant?: 'default' | 'elevated' | 'glass' | 'interactive'  
  padding?: 'none' | 'sm' | 'md' | 'lg'  
}
```

Default: bg var(--surface-2), border 1px var(--border), rounded var(--radius-base)

Elevated: + shadow-md

Glass: bg var(--glass-bg), backdrop-blur-[16px], border var(--glass-border)

Interactive: + hover:bg-[var(--surface-3)] hover:border-[var(--border-strong)]
transition var(--duration-fast), cursor-pointer

Padding: none=0, sm=var(--space-md), md=var(--space-lg), lg=var(--space-xl)

Also export CardHeader, CardContent, CardFooter sub-components
with appropriate spacing.

COMPONENT 4: src/components/ui/badge.tsx

```
interface BadgeProps {  
  severity?: 'critical' | 'high' | 'medium' | 'low' | 'info'  
  variant?: 'severity' | 'status' | 'outline'  
  size?: 'sm' | 'md'  
  children: React.ReactNode  
}
```

Use getSeverityStyles() from utils for severity variant.

Status variant: For scan status badges (completed/scanning/failed/pending)
with animated dot for scanning state.

Outline variant: transparent bg, border only.

sm: text-[11px] px-2 py-0.5, md: text-xs px-3 py-1

Font: weight 600, uppercase, tracking-wider

Border-radius: var(--radius-xs)

COMPONENT 5: src/components/ui/skeleton.tsx

```
interface SkeletonProps {
  className?: string
  variant?: 'text' | 'circular' | 'rectangular'
  width?: string | number
  height?: string | number
}
```

Use the `.skeleton` class from `globals.css` (shimmer animation).

Variants:

```
text: height 16px, rounded var(--radius-sm), width 100%
circular: border-radius 50%
rectangular: rounded var(--radius-md)
```

COMPONENT 6: `src/components/ui/progress-bar.tsx`

```
interface ProgressBarProps {
  value: number // 0-100
  max?: number
  size?: 'sm' | 'md' | 'lg'
  color?: string // CSS color or var()
  showLabel?: boolean
  animated?: boolean
}
```

Track: `bg var(--surface-3), rounded-full`

Fill: `bg accent by default (or custom color), rounded-full,`
`transition width var(--duration-slower) var(--ease-out)`
 Optional shimmer overlay when `animated=true`

`sm: h-1, md: h-2, lg: h-3`

COMPONENT 7: `src/components/ui/tabs.tsx`

```
interface TabsProps {
  tabs: { id: string; label: string; icon?: React.ReactNode }[]
  activeTab: string
  onTabChange: (id: string) => void
  variant?: 'underline' | 'pill'
}
```

Underline: Bottom border indicator, slides to active tab position.

Active: `text-primary`, 2px bottom border accent

Inactive: text-muted, hover text-secondary

Pill: Active has bg surface-3, text-primary. Inactive transparent.

Use framer-motion layoutId for the sliding indicator animation.

Create all 7 component files with full TypeScript implementations.
After creating all components, verify no import errors.

IMPLEMENTATION PROMPT 3: Security-Specific Components

text

Now create the security-specific components unique to Shinodroid.

COMPONENT 1: src/components/security/score-gauge.tsx

```
interface ScoreGaugeProps {  
  score: number // 0-100  
  size?: 'sm' | 'md' | 'lg'  
  animated?: boolean  
  showGrade?: boolean  
}
```

Implementation:

- SVG circle with stroke-dasharray technique
- 270° arc (gap at bottom)
- Track circle: stroke var(--surface-3), same stroke-width
- Fill circle: Colored by score tier (use getScoreTier from utils)
- Stroke-width: sm=6, md=8, lg=10
- Dimensions: sm=64px, md=120px, lg=160px
- Center: Score number (use appropriate text size per size variant)
- Below number: Grade letter (A/B/C/D/F)
- Outer glow: box-shadow colored by tier

Animation (when animated=true):

- Use framer-motion useMotionValue and useTransform
- On mount, animate stroke-dashoffset from full circumference to target
- Simultaneously count up the number from 0
- Duration: 1.2s, ease: [0.16, 1, 0.3, 1]

Calculate:

```
const circumference = 2 * Math.PI * radius
const arcLength = circumference * 0.75 // 270° arc
const fillLength = arcLength * (score / 100)
const dashOffset = arcLength - fillLength
```

SVG transform: rotate(135deg) to start from bottom-left

COMPONENT 2: src/components/security/severity-bar.tsx

```
interface SeverityBarProps {
  critical: number
  high: number
  medium: number
  low: number
  info: number
  height?: number
  showLabels?: boolean
  animated?: boolean
}
```

- Horizontal bar, rounded-full both ends
- Segments proportional to counts
- Colors: critical=#ef4444, high=#f97316, medium=#f59e0b, low=#3b82f6, info=#6b7280
- Count numbers centered in each segment (white text, hide if segment too narrow < 32px)
- Labels below (optional): severity name + count in respective color

Animation: Each segment grows from 0 width with stagger (60ms delay each) using framer-motion.

COMPONENT 3: src/components/security/finding-card.tsx

```
interface FindingCardProps {
  finding: {
    _id: string
    title: string
    severity: string
    category: string
    description: string
    recommendation: string
  }
}
```

```

    cvssScore?: number
    owaspCategory?: string
  }
  defaultExpanded?: boolean
}

```

This is the most complex component. It has collapsed and expanded states.

COLLAPSED (default):

- Card with left accent border (4px solid severity color)
- Severity badge + Title + CVSS pill + Category tag + File path
- Expand chevron (right side)
- onClick: toggle expanded

EXPANDED:

- Same header, chevron rotated 180°
- Divider
- Description section
- Code block (if available – render code with line numbers and highlighted vulnerability line)
- Remediation section (numbered list with accent-colored numbers)
- References section (OWASP and CWE badges)

Use framer-motion AnimatePresence for expand/collapse animation:

```

initial={{ height: 0, opacity: 0 }}
animate={{ height: 'auto', opacity: 1 }}
exit={{ height: 0, opacity: 0 }}
transition={{ duration: 0.25, ease: [0.16, 1, 0.3, 1] }}

```

Code block sub-component:

- Dark bg (#0d0d12)
- Terminal chrome bar (three dots)
- Line numbers column
- Highlighted line with red bg and left accent
- Copy button (top right)

COMPONENT 4: src/components/security/pipeline-step.tsx

```

interface PipelineStepProps {
  label: string
  icon: string // emoji
  status: 'pending' | 'active' | 'completed' | 'failed'
  duration?: string
  progress?: number // 0-100 for active step
}

```

```
  detail?: string // e.g., "Scanning 2,418 / 3,891 files..."
  isLast?: boolean
}
```

Each step renders:

- Connector line (above, except first step)
- Status circle (28px)
- Label + status text
- Connector line (below, except last step)

Connector line colors:

```
completed→completed: solid var(--success)
completed→active: solid var(--success) top, gradient to var(--accent) bottom
active→pending: gradient var(--accent) top to var(--border) bottom, dashed
pending→pending: dashed var(--border)
```

Active step circle: animate-pulse-glow animation

Active step: Show progress bar and detail text

COMPONENT 5: src/components/security/severity-badge.tsx

A thin wrapper around Badge specifically for severity levels.

Adds the colored dot (8px circle) before the text.

Maps severity string to proper colors automatically.

```
interface SeverityBadgeProps {
  severity: 'critical' | 'high' | 'medium' | 'low' | 'info'
  showDot?: boolean
  size?: 'sm' | 'md'
}
```

Create all 5 components with full implementations.

IMPLEMENTATION PROMPT 4: Layout Components

text

Create the shared layout components.

COMPONENT 1: src/components/layout/sidebar.tsx

This REPLACES the visual design of the sidebar in dashboard-shell.tsx but must preserve ALL navigation logic and auth context.

Look at the current dashboard-shell.tsx. Extract:

- The navigation items and their routes
- The user data fetching (useQuery for user info)
- The active route detection logic
- Any auth-related functions (sign out)

Create the new Sidebar component with these exact visuals:

- Width: 260px expanded, 72px collapsed
- Background: var(--surface-1)
- Right border: 1px solid var(--border-subtle)
- Logo at top: Shield icon + "Shinodroid" text (hidden when collapsed)
- Nav items with these routes and icons (use Lucide icons):
 - Dashboard: LayoutDashboard → /dashboard
 - New Scan: Upload → /dashboard/scan
 - Scan History: Clock → /dashboard/reports
 - Settings: Settings → /dashboard/settings
- Active state: bg rgba(124,58,237,0.1), text accent, left border 3px accent
- User section at bottom: Avatar circle with initials + name + plan badge
- Collapse toggle button

Use framer-motion for:

- Width transition on collapse
- Label fade in/out
- Icon scale when collapsed

State: Manage collapsed state internally with useState.

Store preference in localStorage.

Mobile: When viewport < 1024px, sidebar becomes an overlay.

Add a hamburger button that opens it. Use AnimatePresence for slide-in from left with overlay backdrop.

IMPORTANT: Keep all navigation using Next.js Link components.

Keep all existing auth context and useQuery calls.

COMPONENT 2: src/components/layout/top-bar.tsx

```
interface TopBarProps {  
  title?: string // Override greeting  
}
```

- Fetches user name from existing auth context
- Shows time-appropriate greeting: "Good morning/afternoon/evening, [Name] 🤖"
- Right side: Notification bell (static for now) + "New Scan" primary button
- Sticky within content area
- Background: `var(--surface-0)` with bottom border

COMPONENT 3: `src/components/layout/public-nav.tsx`

Navigation for public pages (landing, pricing, docs).

- Sticky top, glass background (blur + semi-transparent)
- Logo left, links center, auth buttons right
- Links: Features, Pricing, Docs (all with smooth scroll or routes)
- "Sign In" ghost button → `/login`
- "Start Free →" primary button → `/signup`
- Mobile: Hamburger menu, slide-in drawer

COMPONENT 4: `src/components/layout/footer.tsx`

Landing page footer with:

- Logo + tagline
- 4 link columns (Product, Resources, Company, Legal)
- Social icons (use Lucide icons: Twitter/X icon, Github, LinkedIn)
- Copyright line
- "Built with 🦋 for the AI-first developer generation"

COMPONENT 5: `src/components/shared/page-transition.tsx`

A wrapper component that adds page enter/exit animations using `framer-motion`.

```
interface PageTransitionProps {
  children: React.ReactNode
  className?: string
}
```

Uses `AnimatePresence` + `motion.div`:

```
initial={{ opacity: 0, y: 8 }}
animate={{ opacity: 1, y: 0 }}
exit={{ opacity: 0, y: -8 }}
transition={{ duration: 0.25, ease: [0.16, 1, 0.3, 1] }}
```

COMPONENT 6: src/components/shared/empty-state.tsx

```
interface EmptyStateProps {
  icon?: React.ReactNode
  title: string
  description: string
  action?: {
    label: string
    href: string
  }
  imageSrc?: string
}
```

Centered layout with optional image/icon, title, description, and optional CTA button.

After creating all layout components, update dashboard-shell.tsx to use the new Sidebar component instead of the old sidebar markup. Keep ALL the existing wrapper logic (ConvexClientProvider, auth checks, etc.) – only replace the JSX for the sidebar visual.

Verify the dashboard still loads correctly after this change.

IMPLEMENTATION PROMPT 5: Chart Components

text

Create the chart components using Recharts.

COMPONENT 1: src/components/charts/score-trend-chart.tsx

```
interface ScoreTrendChartProps {
  data: { name: string; score: number; date: string }[]
  height?: number
}
```

Use Recharts AreaChart:

– Line stroke: var(--accent) #7c3aed, strokeWidth 2

- Area fill: LinearGradient from #7c3aed at 20% opacity to transparent
- Grid: horizontal lines only, stroke #1e1e2e
- X axis: label text 12px, #6b7280, tick line hidden
- Y axis: label text 12px, #6b7280, domain [0, 100]
- Tooltip: Custom tooltip component with:
 - bg var(--surface-4), border 1px var(--border), rounded var(--radius-md)
 - shadow-lg
 - Scan name + score + date
- Dots: fill #7c3aed, stroke #0a0a0f strokeWidth 2, r=4, activeR=6
- ResponsiveContainer width="100%"
- Animate on mount: isAnimationActive={true}

COMPONENT 2: src/components/charts/vuln-donut-chart.tsx

```
interface VulnDonutChartProps {
  critical: number
  high: number
  medium: number
  low: number
  info: number
  size?: number
}
```

Use Recharts PieChart with Pie:

- innerRadius: 60% of size, outerRadius: 80% of size
- Colors: critical=#ef4444, high=#f97316, medium=#f59e0b, low=#3b82f6, info=#6b7280
- Center label: Total count number, 24px weight 700 #e4e4ef
(Use Recharts customized label or absolute positioned div)
- No outline stroke on segments (or very thin #0a0a0f, 1px)
- Hover: activeShape with outerRadius += 4
- Custom tooltip with severity name + count
- Legend: External, positioned below or right:
Colored dot (8px circle) + name + count, stacked vertically
12px, #9ca3b0

```
Animate on mount: startAngle={90} endAngle={-270}
animationDuration={800}
```

Create both chart components with full Recharts implementations.
Make sure they're responsive and handle edge cases
(0 vulnerabilities, single category, etc.)

IMPLEMENTATION PROMPT 6: Page Redesigns (Execute in Order)

text

Now we redesign each page. For EACH page:

1. Read the current page file
2. Extract ALL data fetching logic (useQuery, useMutation, useState, useEffect)
3. Keep that logic EXACTLY as-is
4. Replace ONLY the JSX return block with new markup using our components
5. Test that the page renders correctly

START WITH: Dashboard page (src/app/dashboard/page.tsx)

Current page uses these data sources (DO NOT CHANGE):

- useQuery(api.scans.list) or similar for scan list
- Possibly computed stats from the scan list
- User data from auth context

Replace the current JSX with:

```
<PageTransition>
  <TopBar />

  {/* Stats Row */}
  <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6 mb-8">
    <StatCard title="Total Scans" value={totalScans} trend={...} />
    <StatCard title="Avg Score" value={avgScore} gauge />
    <StatCard title="Critical Issues" value={criticalCount} variant="danger"
  />
    <StatCard title="Monthly Usage" value={monthlyUsage} max={10} progress />
  </div>

  {/* Two Column Layout */}
  <div className="grid grid-cols-1 lg:grid-cols-5 gap-6">
    {/* Left Column - 3/5 */}
    <div className="lg:col-span-3 space-y-6">
      <Card>
        <CardHeader title="Recent Scans" action={<Link>View All </Link>} />
        <RecentScansTable scans={recentScans} />
      </Card>
      <Card>
        <CardHeader title="Security Score Trend" />
        <ScoreTrendChart data={trendData} />
      </Card>
    </div>
  </div>
```

```

    {/* Right Column - 2/5 */}
    <div className="lg:col-span-2 space-y-6">
      <Card>
        <CardHeader title="Vulnerability Distribution" />
        <VulnDonutChart {...vulnCounts} />
      </Card>
      <Card>
        <CardHeader title="Quick Actions" />
        <QuickActions />
      </Card>
      <Card>
        <CardHeader title="Plan Usage" />
        <PlanUsage used={8} total={10} />
      </Card>
    </div>
  </div>

  {/* Empty State (shown when no scans) */}
  {scans.length === 0 && (
    <EmptyState
      title="Welcome to Shinodroid! 🟡"
      description="Upload your first APK to start analyzing it for security
issues"
      action={{ label: "Upload Your First APK", href: "/dashboard/scan" }}
    />
  )}
</PageTransition>

```

Compute all stats (totalScans, avgScore, criticalCount, etc.) from the existing scans data. Do not create new Convex queries.

After dashboard, proceed to each page in this order:

1. Dashboard ← DO THIS ONE FIRST
2. Upload page (scan/page.tsx)
3. Scan detail/report (scan/[id]/page.tsx) – MOST COMPLEX
4. Scan history (reports/page.tsx)
5. Settings (settings/page.tsx)
6. Login (/login/page.tsx)
7. Signup (/signup/page.tsx)
8. Landing page (/page.tsx)

For each page, SHOW ME the full new code before writing it. Wait for my approval before moving to the next page.

CRITICAL FOR scan/[id]/page.tsx:

This page has THREE states based on `scan.status`:

- "pending" or "scanning": Show analysis pipeline + live findings
- "completed": Show full report with tabs
- "failed": Show error state with retry

The existing page already handles these states. KEEP the conditional logic and state management. Only replace the JSX for each state.

The report state (completed) needs:

- Sticky header with ScoreGauge
- Tabs component with three tabs (Overview, Vulnerabilities, Dynamic)
- Overview: Executive summary + stats + severity bar + top findings
- Vulnerabilities: Filter bar + FindingCard list
- Dynamic: Terminal-style output blocks

Use `useQuery(api.findings.listByScan)` for findings – this is already in the code. Do not create new queries.

FOR EACH PAGE: After the redesign, the page must:

- ☐ Load without errors
- ☐ Display all data it displayed before
- ☐ All links and navigation still work
- ☐ All mutations (upload, create scan) still function
- ☐ No console errors related to Convex

IMPLEMENTATION PROMPT 7: Landing Page (New Design)

text

Now redesign the landing page at `src/app/page.tsx`.

This is a PUBLIC page – no auth required, no dashboard sidebar. It uses PublicNav and Footer layout components.

The landing page has 9 sections as specified in DOCUMENT 2. Implement each section in order:

1. PublicNav (sticky, glass effect)
2. Hero Section (full viewport height)
3. Problem Statement (3 problem cards)
4. How It Works (3 steps with connectors)
5. Features Bento Grid (2 large + 4 small cards)
6. Report Preview (screenshot placeholder with callouts)

7. Pricing (3 tier cards with toggle)
8. FAQ (accordion)
9. Final CTA (gradient background)
10. Footer

Animation instructions:

- Each section should fade in when it scrolls into view
- Use framer-motion useInView hook:

```
const ref = useRef(null)
const isInView = useInView(ref, { once: true, margin: "-100px" })
```

```
<motion.div
  ref={ref}
  initial={{ opacity: 0, y: 24 }}
  animate={isInView ? { opacity: 1, y: 0 } : {}}
  transition={{ duration: 0.5, ease: [0.16, 1, 0.3, 1] }}
>
```

- Cards within sections stagger with 60ms delay
- Hero elements stagger with 100ms delay (heading → subheading → buttons → social proof)

For the FAQ accordion, implement with useState tracking openIndex.
Use AnimatePresence for expand/collapse.

For the pricing toggle, use useState for billing period
(monthly/annual). Annual shows 20% discount (calculate discounted prices).

The gradient text effect for the hero headline:
.gradient-text already exists in globals.css – verify it applies
background-clip: text with the gradient vars.

Images: Reference the Canva assets from public/images/ paths.
Use next/image with proper width, height, and alt props.
For placeholder images (not yet created), use colored rectangles
with centered text labels.

Important: This page must NOT import anything from convex/.
It's a pure static/presentational page with no data fetching.
The only interactive elements are:

- Navigation links
- Pricing toggle
- FAQ accordion
- CTA buttons (Link to /signup)

IMPLEMENTATION PROMPT 8: Auth Pages

text

Redesign the login and signup pages.

For `src/app/login/page.tsx`:

Current implementation uses `@convex-dev/auth`. There will be functions like:

- `signIn("password", { email, password })`
- `signIn("github")`
- Or similar auth actions from the Convex auth library

EXTRACT all auth logic:

- form state management (email, password, errors)
- `signIn` function calls
- redirect logic after successful auth
- error handling

KEEP all of that. Only replace the JSX.

New layout: Split-screen (50/50)

LEFT PANEL:

- Full height, bg `surface-0` with subtle radial accent glow
- Auth background pattern image (from `public/images/auth/`)
- Centered: Logo (large) + "by WORMHOLE Security"
- Floating security icons (absolute positioned, low opacity, slow drift animation)
- Bottom: Glass testimonial card

RIGHT PANEL:

- Centered form, max-width 380px
- "Welcome back" heading
- "Sign in to your Shinodroid account" subtitle
- GitHub OAuth button (styled as secondary with GitHub icon)
- "or" divider line
- Email input (with label)
- Password input (with label, show/hide toggle, forgot link)
- Remember me checkbox
- Sign In primary button (full width, shows loading spinner during auth)
- "Don't have an account? Sign up →" link

Error state: Show inline error message below the relevant input.

Use the error state of our Input component.

Loading state: Button shows `isLoading`, inputs get disabled.

Mobile (< 768px): Left panel hidden. Right panel full width.
Small logo + gradient accent at top of form.

For `src/app/signup/page.tsx`:

Same split layout with these differences:

- "Create your account" heading
- Additional "Full Name" input
- Password strength indicator (4-segment bar):
Calculate strength based on length + complexity:
 - < 6 chars: 1/4 (weak, danger color)
 - 6-8 chars: 2/4 (fair, warning color)
 - 8-12 chars with mixed case or numbers: 3/4 (good, info color)
 - 12+ chars with mixed case + numbers + symbols: 4/4 (strong, success color)
- "Create Account →" CTA
- Terms text below
- Different testimonial

PRESERVE ALL AUTH LOGIC. Only change the visual presentation.

After implementing, test:

- ☐ Login with email/password works
- ☐ GitHub OAuth button triggers the flow
- ☐ Error messages display properly
- ☐ Redirect after login works
- ☐ Signup creates account successfully
- ☐ Password strength indicator updates as user types

IMPLEMENTATION PROMPT 9: Polish & Micro-Interactions

text

Final polish pass. Add these animations and refinements to all pages:

1. PAGE TRANSITIONS:

Wrap all page content in `<PageTransition>` component.

This adds `fadeInUp` on mount to every page.

2. STAT NUMBER COUNT-UP:

In dashboard stat cards, animate numbers counting up from 0:

Use framer-motion useMotionValue + useTransform + animate:

```
function AnimatedNumber({ value }: { value: number }) {
  const count = useMotionValue(0)
  const rounded = useTransform(count, Math.round)

  useEffect(() => {
    const animation = animate(count, value, {
      duration: 1.2,
      ease: [0.16, 1, 0.3, 1],
    })
    return animation.stop
  }, [count, value])

  return <motion.span>{rounded}</motion.span>
}
```

3. SKELETON LOADING:

For every page that fetches data, show skeletons while loading:

Dashboard: 4 skeleton stat cards + skeleton table rows + skeleton chart

Upload: Not needed (no data fetch)

Report: Skeleton header + skeleton content blocks

History: Skeleton rows

Settings: Skeleton form fields

Use the Skeleton component with appropriate widths/heights matching the final content layout.

Detection: If useQuery returns undefined, show skeleton.

If it returns data, show real content.

4. BUTTON HOVER EFFECTS:

All primary buttons: On hover, add subtle scale(1.02) and glow-sm.

On active: scale(0.98). Transition: var(--duration-fast).

5. CARD HOVER EFFECTS:

Interactive cards (scan history rows, quick action items):

On hover: translateY(-2px) + shadow-md. Transition: var(--duration-fast).

6. SIDEBAR ACTIVE INDICATOR:

When route changes, the active indicator should animate to new position.

Use framer-motion layoutId="sidebar-active" on the active background.

7. TOAST NOTIFICATIONS (OPTIONAL BUT RECOMMENDED):

Create a simple toast system for:

- "Scan started successfully" (success)
- "Upload failed" (error)
- "Report downloaded" (success)

Use a context provider + AnimatePresence:

- Toast slides in from top-right
- Auto-dismisses after 5 seconds
- Has a shrinking progress bar at bottom
- Swipe/click to dismiss early

8. SCROLL PROGRESS (Landing page):

Thin line at the very top of the viewport (2px) showing scroll progress.
Color: accent gradient. Goes from 0% to 100% width as user scrolls.

9. PREFERS-REDUCED-MOTION:

Add this check to disable all animations when user prefers:

```
const prefersReducedMotion =  
  typeof window !== 'undefined' &&  
  window.matchMedia('(prefers-reduced-motion: reduce)').matches
```

```
Pass to framer-motion: transition={{ duration: prefersReducedMotion ? 0 : 0.25  
}}
```

10. FOCUS VISIBLE STYLES:

All interactive elements must show visible focus ring for keyboard nav:

```
focus-visible: ring-2 ring-[var(--accent)] ring-offset-2 ring-offset-[var(--  
surface-0)]
```

Implement all of these. After each, verify no regressions.